



對話交接文件

1. 主題與目標

核心主題

建立一個個人使用的投資研究網頁系統，第一個實作功能為：

> 台股族群成交值排行榜，以及 5/10/20/60 個交易日的排行與成交熱度變化。

本次對話依據使用者提供的 `Invest大綱(1).pdf` 接續規劃。附件記載過去已完成的需求整理、架構討論、指標公

使用者目前明確要求

1. 專案需從零開始教學，將使用者視為網頁開發新手。
2. 使用者有 N 個未來待開發的投資功能，但目前只先做「成交值排行」。
3. 功能與程式目錄必須事先規劃，不可隨意放置檔案。
4. 專案固定放在：

```
```text
/Users/frankchiang/Desktop/Application/Code/Project/Invest
```
```

5. 最終成果必須是可互動的網頁。
6. 開發必須採小步驟進行，不可一次加入大量技術。
7. 第一階段先完成可執行的最小版本，不先處理漂亮圖表或完整投資系統。

第一版預期成果

第一個可運作版本預計包含：

- 台股族群成交值排行榜。
- 期間切換。
- 排名與排名變化。
- 成交值增減。
- 市場成交比。
- 上漲家數比。
- 前三大成交集中度。
- 成分股成交值貢獻。
- 點擊族群進入明細。
- 表格排序等網頁互動。

第一版不是盤中即時系統，而是以日線／收盤後資料為核心。

2. 使用者背景

工作與技術背景

使用者是軟體工程師，約有 8 年工作經驗。

主要經驗：

- C#。
- WPF。
- Windows。
- Visual Studio。
- 少部分 C++。
- 機器視覺。
- 自動化應用。
- SPI／PCB 檢測設備相關系統。

本專案開發環境

個人開發環境為：

- macOS。
- VS Code。
- 專案開發位置為 Mac 本機。

熟悉與不熟悉的範圍

使用者熟悉：

- C# 語法。
- 物件導向。
- WPF。
- Visual Studio 的專案與除錯概念。

使用者對以下流程不熟悉，需要用新手方式說明：

- macOS Terminal。
- CLI 指令。
- VS Code 的 C# 開發流程。
- ASP.NET Core。
- Blazor。
- Entity Framework Core。
- SQLite。
- Migration。
- Google Cloud。
- Google Sheets API 認證。
- Git 指令流程。
- 網頁前後端架構。
- 網頁專案部署。

教學方式要求

每個操作步驟應說明：

1. 指令在哪裡輸入。

2. 指令的用途。
3. 預期會看到什麼。
4. 檔案要建立在哪個路徑。
5. 程式碼要貼入哪個檔案。
6. 執行失敗時，需要提供哪些錯誤資訊。
7. 應等待使用者貼出結果後，再繼續下一階段。

3. 現況摘要

需求與架構現況

目前尚未真正建立專案或撰寫程式。

已完成的內容主要是：

- 分析原始 Google Sheets 的限制。
- 討論過完整投資知識庫架構。
- 決定暫緩概念股與知識圖譜。
- 將第一優先功能收斂為族群成交值排行榜。
- 決定以 C# 為主要技術。
- 決定第一版使用 Blazor Web App。
- 決定先用測試資料驗證公式。
- 規劃未來使用 SQLite 保存歷史資料。
- 規劃 Google Sheets 與資料庫的責任分工。
- 初步設計資料表。
- 初步規劃功能目錄與開發順序。

尚未執行的事項

目前尚未執行：

- 尚未確認 Mac 的 .NET SDK 環境。
- 尚未建立 `Invest.slnx`。
- 尚未建立 `Invest.Web`。
- 尚未執行 Blazor 空白網頁。
- 尚未建立 Git Repository。
- 尚未建立測試資料。
- 尚未撰寫成交值排行計算。
- 尚未建立 SQLite。
- 尚未安裝 EF Core。
- 尚未建立 Migration。
- 尚未串接 Google Sheets。
- 尚未串接 TWSE。
- 尚未串接 TPEx。
- 尚未回補歷史行情。
- 尚未建立任何正式網頁畫面。

本次對話最後停留位置

上一個回覆已提出「階段 0：環境檢查」，請使用者在 Terminal 執行以下指令並貼回完整結果：

```
```bash
pwd
sw_vers -productVersion
uname -m
dotnet --version
dotnet --list-sdks
git --version
code --version
```
```

目前使用者尚未貼回上述執行結果。

4. 已確認事實

使用情境

- 第一階段是使用者個人使用。
- 在 Mac 本機開發。
- 現階段不需要多人帳號系統。
- 現階段不是公開商業網站。
- 主要分析台股上市及上櫃股票。
- 核心分析頻率是日線／收盤後。
- 第一版不處理盤中即時行情。
- 最終介面是互動式網頁。
- 第一個功能是族群成交值排行。

專案位置

固定專案根目錄：

```
```text
/Users/frankchiang/Desktop/Application/Code/Project/Invest
```
```

資料來源方向

未來正式行情資料來源方向：

- 上市股票：臺灣證券交易所 TWSE 官方資料。
- 上櫃股票：證券櫃檯買賣中心 TPEX 官方資料。

實際 Endpoint 尚未選定及驗證。

Google Sheets 的定位

過渡期間仍保留 Google Sheets，主要管理相對靜態且由人工維護的資料：

- 族群名稱。
- 族群包含哪些股票。

- 股票代號。
- 股票名稱。
- 是否啟用。
- 族群備註。

Google Sheets 可視為：

> 族群定義及成分股管理介面。

先前建議格式：

| GroupKey | 族群名稱 | 股票代號 | 股票名稱 | 啟用 |
|-------------------------|------|------|------|------|
| --- --- --- --- --- | | | | |
| advanced-packaging | 先進封裝 | 3131 | 弘塑 | TRUE |
| advanced-packaging | 先進封裝 | 3583 | 辛耘 | TRUE |
| facility-engineering | 廠務工程 | 2404 | 漢唐 | TRUE |

Google Sheets 的實際欄位格式目前尚未整理完成。

SQLite 的定位

所有會隨時間累積的資料，未來應放入 SQLite：

- 每日個股行情。
- 每日個股成交值。
- 每日族群統計。
- 每日族群指標。
- 每日成分股貢獻。
- 歷史排名所需資料。
- 資料匯入紀錄。
- 當日族群成員狀態。
- Membership Hash。
- 計算時間。

簡化分工：

> Google Sheets 定義目前族群包含哪些股票；SQLite 記錄每天市場發生了什麼，以及當時使用哪些族群成員計算

歷史一致性要求

歷史結果必須保持穩定。

問題情境：

1. 今天將一檔股票加入「機器人」族群。
2. 系統若每次查詢歷史資料時，都使用今天最新的成分股名單。
3. 過去幾個月的族群成交值與排行會被改寫。
4. 相同日期的歷史結果可能每次查詢都不同。

因此每日計算完成後，應保存：

- 每日族群指標。

- 每日成分股貢獻。
- 當日成分股數量。
- 當日成分股清單 Hash。
- 計算時間。

不應在每次開啟網頁時，使用目前最新族群成員重算全部歷史。

族群重疊原則

- 同一股票可以屬於多個族群。
- 同一族群內是否允許重複股票，尚待定義去重規則。
- 由於族群會重疊，各族群市場成交比加總可能超過 100%。
- 加總超過 100% 不一定是錯誤。
- UI 未來應提醒使用者，族群市場成交比不可直接加總解讀。

5. 已完成事項

已完成的需求整理

已釐清原始 Google Sheets 的問題：

1. 族群與概念股難以系統化連結。
2. 同一公司可能屬於多個族群或概念。
3. 同一新聞可能關聯多個公司及族群。
4. 多個股票或多個網址放在同一儲存格，難以查詢、統計與維護。
5. Google Sheets 不適合大量時間序列資料與複雜歷史查詢。

已完成的範圍收斂

原本討論過完整投資研究知識庫，包括：

- Company。
- Category。
- CompanyCategory。
- CategoryRelations。
- NewsEvent。
- NewsSource。
- NewsCategory。
- NewsCompany。
- ResourceLink。
- LegacyOverrides。

但目前已暫緩：

- 概念股知識圖譜。
- 完整產業供應鏈地圖。
- 族群新聞時間軸。
- 多來源新聞連結管理。
- AI 新聞摘要。
- 交易紀錄 Dashboard。

- 資產配置 Dashboard。
- 盤中即時行情。
- 手機 App。
- 買賣訊號。
- AI 綜合評分。

目前只優先處理：

> 台股族群成交值排行榜與歷史排行變化。

已完成的技術方向規劃

第一版技術方向已規劃為：

- macOS。
- VS Code。
- C# Dev Kit。
- .NET 10。
- Blazor Web App。
- Interactive Server。
- Global Interactivity。
- ASP.NET Core。
- Entity Framework Core。
- SQLite。
- Google Sheets API。
- Git。
- GitHub 私有 Repository。

已提出的專案架構

建議使用：

```
```text
一個 Git Repository
一個 Solution
一個 Blazor Web Project
多個 Features 功能目錄
```
```

不為每一項投資功能建立獨立 ``.csproj``。

「N 個待執行專案」在目前架構中，建議理解為 N 個業務功能模組，例如：

- TradingValueRanking。
- Portfolio。
- TradingRecords。
- News。

目前只建立 ``TradingValueRanking``，其他功能等實際開發時再建立。

已提出的專案目錄

```
```text
```

```
Invest/
```

```
|─ Invest.slnx
```

```
|─ README.md
```

```
|─ .gitignore
```

```
|
```

```
|─ docs/
```

```
| |─ architecture/
```

```
| |─ decisions/
```

```
| |─ backlog/
```

```
|
```

```
|─ data/
```

```
| |─ samples/
```

```
| |─ imports/
```

```
| |─ database/
```

```
|
```

```
|─ scripts/
```

```
|
```

```
|─ src/
```

```
| |─ Invest.Web/
```

```
| |
```

```
| | |─ Components/
```

```
| | | |─ Layout/
```

```
| | | |─ Pages/
```

```
| | | |─ Shared/
```

```
| |
```

```
| | |─ Features/
```

```
| | | |─ TradingValueRanking/
```

```
| | | | |─ Components/
```

```
| | | | |─ Models/
```

```
| | | | |─ Services/
```

```
| | | | |─ SampleData/
```

```
| |
```

```
| | |─ Domain/
```

```
| | | |─ Stocks/
```

```
| | | |─ StockGroups/
```

```
| |
```

```
| | |─ Data/
```

```
| | | |─ Configurations/
```

```
| | | |─ Migrations/
```

```
| |
```

```
| | |─ Infrastructure/
```

```
| | | |─ Persistence/
```

```
| | | |─ GoogleSheets/
```

```
| | | |─ MarketData/
```

```
| | | | |─ Twse/
```

```
| | | | |─ Tpex/
```

```
| |
```

```
| | |─ wwwroot/
```

```
| | |─ Program.cs
```

```
| | |─ appsettings.json
```

```
| | |─ Invest.Web.csproj
```

```
|
```



```
└─ tests/
 └─ Invest.Web.Tests/
 ...
```

此目錄尚未實際建立。

---

## ## 6. 重要決策與理由

### ### 決策一：先做成交值排行

原因：

- 使用者目前最需要判斷哪些族群正在出現成交熱度。
- 成交值排行的輸入、計算及輸出較明確。
- 每日歷史資料累積後，系統價值會自然提高。
- 相較完整知識圖譜，第一版開發範圍較可控。

### ### 決策二：暫緩概念股與知識圖譜

原因：

- 市場已有其他產業地圖網站可供參考。
- 關聯管理範圍大。
- 第一版成本較高。
- 與目前最急迫需求無直接關係。

參考網站：

```
```text
https://aistockmap.com/?topic=advanced-packaging-test&activeTab=structure
```
```

該網站僅作為頁面與產業架構參考，不代表要複製其資料或功能。

### ### 決策三：第一版做日線／收盤後資料

原因：

- 需要比較 5／10／20／60 個交易日趨勢。
- 盤中即時資料對第一版的核心價值有限。
- 即時資料會額外增加：
  - 資料授權問題。
  - 更新頻率管理。
  - WebSocket。
  - 快取。
  - 大量寫入。
  - 錯誤補償。
  - API 穩定性問題。

### ### 決策四：使用 C# 為主的網頁技術

原因：

- 使用者已有 C# 經驗。
- 不需同時學習 React、TypeScript 與 Node.js。
- 前端互動與後端資料處理可使用同一語言。
- Google API 憑證可留在 Server 端。

目前選擇：

```
```text
.NET 10
Blazor Web App
Interactive Server
Global Interactivity
ASP.NET Core
```
```

不使用 WebAssembly 作為第一版主要模式。

### 決策五：第一版單一 Solution、單一 Web Project

原因：

- 個人、本機使用。
- 第一版功能範圍有限。
- 避免一開始導入過度複雜的 Clean Architecture。
- 避免多專案參考與部署複雜度。
- 仍透過功能目錄維持邏輯分區。

### 決策六：以功能模組為主要目錄單位

成交值排行相關檔案集中於：

```
```text
src/Invest.Web/Features/TradingValueRanking
```
```

該功能下再分：

```
```text
Components
Models
Services
SampleData
```
```

目的：

- 避免頁面、計算、模型與假資料散落各處。
- 未來新增其他投資功能時可使用相同結構。
- 不必為每個功能建立獨立 ``.csproj``。

### 決策七：先使用測試資料，不立即串正式 API

第一階段先使用：

- 程式內建資料。
- JSON。
- 或 CSV。

驗證：

- 期間切割。
- 本期及前期比較。
- 平均成交值。
- 排名。
- 排名變化。
- 市場成交比。
- 上漲家數比。
- 集中度。
- 成分股貢獻。

原因：

> 若公式、資料庫與外部 API 同時開發，發生錯誤時很難判斷問題位於資料來源、資料轉換、計算還是 UI。

### 決策八：先做表格，不先做圖表

第一版優先：

- 排行表。
- 期間按鈕。
- 排序。
- 族群明細。
- 成分股貢獻。

後續才加入：

- 折線圖。
- 熱力圖。
- 趨勢圖。
- Dashboard。

### 決策九：先用 SQLite，不先用 PostgreSQL

原因：

- 單人本機使用。
- 不需啟動資料庫伺服器。
- 資料可保存成單一檔案。
- 適合 macOS。
- 可搭配 Entity Framework Core。
- 未來若需外部部署，再評估 PostgreSQL。

### 決策十：Google Sheets 與資料庫不可雙向管理同一筆資料

原則：

- Google Sheets 暫時負責族群及成分股定義。
- SQLite 負責歷史行情及每日計算結果。
- 不應讓相同資料同時在 Google Sheets 與 SQLite 雙向修改。

---

## ## 7. 技術細節

### ### 7.1 目錄責任

| 目錄 | 責任 |

| --- | --- |

| `Components/Pages` | 完整網頁頁面及路由 |

| `Components/Layout` | 主選單、側邊欄、整體版面 |

| `Components/Shared` | 跨功能共用 UI 元件 |

| `Features/TradingValueRanking/Components` | 期間按鈕、排行榜表格、族群明細元件 |

| `Features/TradingValueRanking/Models` | 排行查詢結果、頁面 ViewModel |

| `Features/TradingValueRanking/Services` | 期間切割、統計、排名與比較邏輯 |

| `Features/TradingValueRanking/SampleData` | 第一階段 JSON/CSV 測試資料 |

| `Domain/Stocks` | 股票核心模型 |

| `Domain/StockGroups` | 族群及族群成員核心模型 |

| `Data` | EF Core、DbContext、Configuration、Migration |

| `Infrastructure/GoogleSheets` | Google Sheets 外部資料讀取 |

| `Infrastructure/MarketData/Twse` | TWSE 外部資料來源 |

| `Infrastructure/MarketData/Tpex` | TPEx 外部資料來源 |

| 根目錄 `data` | SQLite 檔案、匯入檔、測試資料檔 |

| `docs` | 架構、決策紀錄、待辦事項 |

| `tests` | 公式及服務單元測試 |

已確定可保留的名稱：

- `AppDbContext`
- `GoogleSheetsClient`

其餘 Service、Repository 與 DTO 名稱尚未定案。

### ### 7.2 Blazor 模式

預計建立：

```text

Blazor Web App

Interactivity: Server

Interactivity location: Global

Authentication: None

```

架構概念：

```
```text
```

瀏覽器

↓

Blazor Server / ASP.NET Core

├─ 排行計算服務

├─ Google Sheets API

├─ TWSE 資料來源

├─ TPEX 資料來源

└─ SQLite

```
```
```

### ### 7.3 預計網頁路由

上一個回覆提議第一版使用：

```
```text
```

/trading-value-ranking

```
```
```

此路由名稱尚未經使用者再次確認，但可作為合理預設。

### ### 7.4 成交熱度排行

回答的問題：

> 最近哪個族群吸收最多成交值？

公式：

```
```text
```

近 N 日平均每日成交值

= 近 N 個交易日族群總成交值 $\div$ 實際交易日數

```
```
```

不應只看單日總成交值。

### ### 7.5 資金加速排行

回答的問題：

> 哪個族群近期成交值，相較前一個同長度期間快速放大？

以 5 日為例：

```
```text
```

本期：最近 5 個交易日

前期：再往前 5 個交易日

```
```
```

成交值倍數：

```text

成交值倍數

= 本期平均每日成交值 ÷ 前期平均每日成交值

```

成交值增減率：

```text

成交值增減率

= (本期平均每日成交值 - 前期平均每日成交值)
÷ 前期平均每日成交值

```

需處理前期成交值為 0 的情況，實際規則尚未設計。

### ### 7.6 排名變化

公式：

```text

排名變化

= 前期排名 - 本期排名

```

範例：

```text

前期排名：第 8 名

本期排名：第 1 名

排名變化 = 8 - 1 = +7

```

解讀：

- 正數：排名上升。
- 負數：排名下降。
- 0：排名不變。

### ### 7.7 市場成交比

公式：

```text

族群市場成交比

= 族群成交值 ÷ 全市場股票成交值

```

理由：

若族群成交值增加 30%，但整體市場成交值增加 50%，該族群取得的市場注意力實際上可能下降。

應同時觀察：

- 絕對成交值。
- 市場成交比。
- 市場成交比的期間變化。

### ### 7.8 上漲家數比

原始公式方向：

```text

上漲家數比

= 期間內上漲的有效成分股數
÷ 有效行情資料的成分股數

```

用途：

- 判斷是否整個族群同步上漲。
- 判斷是否僅少數大型股票帶動。

但「期間內上漲」的正式定義尚未決定，可能是：

- 期間起點收盤價到終點收盤價。
- 每日上漲天數。
- 期間平均日漲跌。

上一個回覆暫時提議第一版採「期間終點相較起點」，但尚未得到使用者確認。

### ### 7.9 前三大成交集中度

公式：

```text

前三大成交集中度

= 族群內成交值最高的前三檔股票成交值合計
÷ 族群總成交值

```

解讀：

- 高集中度：可能只有少數股票在動。
- 低集中度：資金較廣泛擴散至族群成分股。

若族群有效成分股少於三檔，實際處理規則尚未明確定義。

### ### 7.10 成分股成交值貢獻

族群明細頁預計顯示：

- 股票代號。

- 股票名稱。
- 近 N 日成交值。
- 占族群成交值比例。
- 期間漲跌。
- 成交值增減。

### ### 7.11 第一版排行表欄位草案

附件原始草案：

- 排名。
- 排名變化。
- 族群。
- 近 N 日平均成交值。
- 較前 N 日成交值增減率。
- 市場成交比。
- 市場成交比變化。
- 族群期間漲跌。
- 上漲家數比。
- 前三大成交集中度。

上一個回覆另提議加入：

- 成分股數。

「族群期間漲跌」因計算方式尚未定案，第一版可能先不顯示。

### ### 7.12 第一版互動功能草案

上一個回覆提出：

1. 期間切換。
2. 點擊欄位標題排序。
3. 切換排行模式。
4. 點擊族群名稱進入明細。
5. 顯示成分股成交值貢獻排行。
6. 顯示載入中狀態。
7. 顯示資料不足狀態。

排行模式暫定：

```
```text
成交熱度
資金加速
```
```

暫定排序：

```
```text
成交熱度：近 N 日平均每日成交值
資金加速：較前 N 日成交值增減率
```
```



上述是上一個回覆為了推進 MVP 提出的暫定規格，尚未得到使用者明確確認。

### ### 7.13 測試資料規模草案

附件建議第一版測試資料：

- 3 個測試族群。
- 每個族群 5~10 檔股票。
- 約 20~30 個交易日。
- 期間切換：5/10/20 日。

上一個回覆提議：

- 使用約 30 個交易日。
- 第一個假資料版本先開放 5/10/20 日。
- 60 日需要至少 120 個交易日，待歷史資料增加後再開放。

此規格尚未由使用者正式確認。

### ### 7.14 初步資料表

#### #### Stock

```
```text
Id
Market
Ticker
Name
IsActive
```
```

唯一條件：

```
```text
Market + Ticker
```
```

#### #### StockGroup

```
```text
Id
GroupKey
Name
Description
IsActive
```
```

#### #### StockGroupMember

```
```text
Id
```

```
StockGroupId
StockId
EffectiveFrom
EffectiveTo
Source
...
```

用途：

- 表達股票屬於哪個族群。
- 支援成員關係生效與失效日期。
- 記錄資料來源。

```
#### DailyStockTrading
```

```
```text
TradingDate
StockId
OpenPrice
HighPrice
LowPrice
ClosePrice
ChangePercent
TradingVolume
TradingValue
TransactionCount
...
```

核心欄位：

```
```text
TradingDate
StockId
ClosePrice
TradingValue
...
```

```
#### DailyGroupMetric
```

```
```text
TradingDate
StockGroupId
TotalTradingValue
MarketShare
GroupReturn
AdvancingRatio
Top3Concentration
MemberCount
MembershipHash
CalculatedAt
...
```

用途：

- 保存每日族群計算結果。
- 保存當日成員版本。
- 避免歷史結果漂移。

#### DailyGroupStockContribution

```text

TradingDate
StockGroupId
StockId
TradingValue
ContributionRatio
ReturnPercent
```

用途：

- 保存每天每個成分股的成交值貢獻。
- 回查某日期由哪些股票帶動族群。

#### 資料匯入紀錄

資料表名稱與欄位尚未正式決定，預計至少包含：

```text

資料日期
資料來源
匯入時間
匯入筆數
是否成功
錯誤訊息
```

### 7.15 Google Sheets API 認證方向

未來預計：

1. 建立 Google Cloud Project。
2. 啟用 Google Sheets API。
3. 建立 Service Account。
4. 將 Google Sheet 分享給 Service Account 電子郵件。
5. 權限先給 Viewer。
6. 憑證只放在 Server 端。
7. 不得將憑證 Commit 至 GitHub。
8. 使用 `dotnet user-secrets` 或安全本機設定保存憑證路徑。

此部分尚未執行。

### 7.16 Google Sheets 讀取策略

不應每次切換畫面都呼叫 Google Sheets。

建議流程：

```
```text
程式啟動或手動刷新
      ↓
一次讀取需要的 Sheet／Range
      ↓
轉換成 C# Model
      ↓
放入記憶體快取
      ↓
網頁查詢快取資料
```
```

第一版可先：

- 啟動時讀取一次。
- 提供「重新整理 Google Sheets」按鈕。
- 不做高頻背景同步。

### 7.17 預計專案建立指令

上一個回覆規劃在環境確認後執行：

```
```bash
dotnet new sln -n Invest
```
```

預期在 .NET 10 環境建立：

```
```text
Invest.slnx
```
```

接著建立 Blazor Web App：

```
```bash
dotnet new blazor \
  -n Invest.Web \
  -o src/Invest.Web \
  -f net10.0 \
  --interactivity Server \
  --all-interactive \
  --auth None \
  --empty
```
```

上述指令目前尚未執行。

### 7.18 環境檢查指令

已要求使用者執行：

```
```bash
mkdir -p "/Users/frankchiang/Desktop/Application/Code/Project/Invest"

cd "/Users/frankchiang/Desktop/Application/Code/Project/Invest"

pwd

sw_vers -productVersion

uname -m

dotnet --version

dotnet --list-sdks

git --version

code --version
```
```

目的：

- 確認專案路徑。
- 確認 macOS 版本。
- 確認 Apple Silicon 或 Intel。
- 確認 .NET SDK。
- 確認 Git。
- 確認 VS Code 的 `code` 指令。

目前尚未取得執行結果。

---

## 8. 尚未解決問題

### 產品定義

1. 首頁預設期間尚未正式決定：

- 5 日。
- 10 日。
- 20 日。

2. 是否需要 1 日排行尚未決定。

3. 第一排序預設應為何者尚未最終決定：

- 絕對成交值。
- 成交值增幅。
- 市場成交比增幅。
- 其他方式。

4. 是否需要「族群在動」綜合條件尚未定義。
5. 目前不建議一開始建立不透明的 AI 或綜合分數。
6. 第一版是否同時顯示 60 日尚未確認。
  - 若使用本期 60 日對比前期 60 日，至少需要 120 個交易日。

### ### 指標算法

7. 族群期間報酬率尚未確認：
  - 等權平均。
  - 成交值加權。
  - 市值加權。
  - 族群指數。
  - 其他方式。
8. 股票停牌、無成交、新上市時，如何處理尚未決定。
9. 上漲家數比的期間定義尚未決定：
  - 起點至終點漲跌。
  - 每日上漲天數。
  - 期間平均日漲跌。
10. 前期比較是否永遠採緊鄰的同長度期間，尚未由使用者再次確認。
11. 前期平均成交值為 0 時，增減率與倍數如何顯示尚未設計。
12. 排名相同時使用何種次排序規則尚未決定。
13. 有效成分股不足三檔時，前三大集中度的計算規則尚未正式決定。
14. 成交值顯示單位尚未決定：
  - 元。
  - 萬元。
  - 億元。

### ### 族群資料

15. Google Sheets 的族群資料實際欄位與排列尚未整理。
16. 同一股票重複出現在同一族群時，需要建立去重或錯誤規則。
17. 族群名稱修改時，`GroupKey` 是否維持不變尚未正式確認。
18. 族群成分股生效日期由 Google Sheets 維護，或由同步程式產生，尚未決定。
19. Google Sheets 中需要讀取的 Sheet 名稱及 Range 尚未確定。

### ### 行情資料

20. TWSE 實際 Endpoint 尚未選定。

21. TPEx 實際 Endpoint 尚未選定。

22. 需確認官方資料能否取得：

- 開盤價。
- 最高價。
- 最低價。
- 收盤價。
- 成交股數。
- 成交金額。
- 成交筆數。
- 漲跌幅。

23. 歷史資料回補方式尚未確認。

24. 第一版正式歷史資料回補天數尚未決定，附件建議 120~250 個交易日。

25. 缺漏日期、API 失敗與重複匯入的處理方式尚未設計。

26. 上市與上櫃是否合併為同一市場總成交值，或可分市場比較，尚未正式決定。

### ### 系統與部署

27. 正式產品名稱尚未決定。

28. GitHub Repository 尚未建立。

29. SQLite 資料庫檔案最終路徑尚未決定。

- 目前目錄規劃包含根目錄 `data/database/`，但尚未正式確認。

30. 未來是否需要手機或外部網路存取尚未決定。

31. 每日資料更新的觸發方式尚未決定：

- 手動按鈕。
- 程式啟動時。
- `BackgroundService`。
- macOS 排程。

第一版較適合先使用手動更新，但尚未實作。

32. 尚未確認使用者 Mac 的：

- macOS 版本。
- CPU 架構。
- .NET SDK。
- Git。
- VS Code `code` 指令。

---

## ## 9. 限制、偏好與注意事項

### ### 回答語言與形式

- 使用繁體中文。
- 教學需明確且具體。
- 不要只講概念。
- 指令需逐行說明。
- 程式碼需標明檔名及完整路徑。
- 使用者雖然有 C# 經驗，但網頁開發部分應視為新手。
- VS Code 操作可適度對照 Visual Studio 概念。
- 每次不要導入過多新技術。
- 優先讓程式成功執行，再解釋進階架構。

### ### 技術限制

第一版不要加入：

- React。
- Next.js。
- Vue。
- TypeScript。
- Docker。
- PostgreSQL。
- Redis。
- Kubernetes。
- 微服務。
- 獨立前後端專案。
- 複雜 Clean Architecture。
- 盤中即時行情。
- AI 綜合評分。
- 新聞系統。

### ### 架構注意事項

- 不要將所有 C# 類別直接放在專案根目錄。
- 不要將業務計算寫入 Razor 頁面。
- 不要將外部 API 程式寫入 `Domain`。
- 不要讓 UI 直接操作 EF Core。
- 不要在第一版建立大量空專案。
- 不要為每個功能建立獨立微服務。
- 功能模組應集中在 `Features` 目錄。
- 共用核心模型才放入 `Domain`。
- 外部系統整合才放入 `Infrastructure`。

### ### 資料原則

優先順序：



1. 計算正確。
2. 資料可追溯。
3. 歷史結果穩定。
4. UI 可查詢。
5. 最後才是視覺效果。

其他原則：

- 不要讓 Google Sheets 與資料庫雙向管理同一筆資料。
- 歷史資料不可因目前族群名單改變而漂移。
- 同一股票可以屬於多個族群。
- 族群成交比不可假設總和為 100%。
- 尚未確定的算法不可自行宣稱已定案。
- 外部 API 串接前，先使用假資料驗證公式。
- 不要宣稱目前已完成任何程式、資料庫或 API；目前仍停留在規劃及環境檢查前。

---

## ## 10. 相關檔案與連結

### ### 使用者上傳附件

```
```text
檔名：Invest大綱(1).pdf
本機路徑：/mnt/data/Invest大綱(1).pdf
頁數：28 頁
```
```

用途：

- 過去對話的交接文件。
- 記錄需求、決策、技術方向、資料表草案及建議開發步驟。
- 本次專案規劃的主要依據。

### ### 專案目錄

```
```text
/Users/frankchiang/Desktop/Application/Code/Project/Invest
```
```

### ### 原始 Google Sheets

```
```text
https://docs.google.com/spreadsheets/d/1rZpWz5GSHkvjoXlp5mX6SKoe2Yhn0MMfNfiiqTpDpZI/edit?
```
```

附件中記載曾辨識到的分頁包括：

- 每日新聞。
- 故事。
- 上市櫃名單 & 族群。
- 概念股。

- 筆記。
- 出貨旺季。
- 操作（台）。
- 操作（台）\_韭菜。
- 損益—操作（台）。
- 損益—存股（台）。
- 損益—存股（美）。
- 資產配置。
- 投信持有率。
- 待辦事項。

目前成交值排行第一版主要會使用其中的族群及成分股定義，實際分頁與欄位仍需確認。

### ### 產業結構參考網站

```text

<https://aistockmap.com/?topic=advanced-packaging-test&activeTab=structure>

```

用途：

- 參考產業／族群結構呈現。
- 不代表要複製資料或完整功能。

### ### 官方行情來源

臺灣證券交易所：

```text

<https://www.twse.com.tw/>

```

TWSE OpenAPI：

```text

<https://openapi.twse.com.tw/>

```

證券櫃檯買賣中心：

```text

<https://www.tpex.org.tw/>

```

TPEx OpenAPI：

```text

<https://www.tpex.org.tw/openapi/>

```

實際 Endpoint 尚未選定與驗證。

---

## ## 11. 建議下一步

### ### 下一步一：完成環境檢查

先不要建立正式功能程式。

請使用者在 macOS Terminal 執行：

```
```bash
mkdir -p "/Users/frankchiang/Desktop/Application/Code/Project/Invest"

cd "/Users/frankchiang/Desktop/Application/Code/Project/Invest"

pwd

sw_vers -productVersion

uname -m

dotnet --version

dotnet --list-sdks

git --version

code --version
```
```

請使用者貼回完整輸出。

檢查目標：

- `pwd` 是否為正確專案目錄。
- Mac 是 `arm64` 或 `x86\_64`。
- 是否已安裝 .NET 10 SDK。
- 是否已安裝 Git。
- VS Code 是否可使用 `code` 指令。

如果 `dotnet` 不存在，應先依 CPU 架構協助安裝正確 SDK。

如果 `code` 不存在，需在 VS Code 執行：

```
```text
Command Palette
→ Shell Command: Install 'code' command in PATH
```
```

### ### 下一步二：建立 Solution 與空白 Blazor 專案

環境確認後，預計執行：

```
```bash
cd "/Users/frankchiang/Desktop/Application/Code/Project/Invest"

dotnet new sln -n Invest

mkdir -p src tests docs data scripts

dotnet new blazor \
  -n Invest.Web \
  -o src/Invest.Web \
  -f net10.0 \
  --interactivity Server \
  --all-interactive \
  --auth None \
  --empty
```
```

接著將 Web Project 加入 Solution。

加入 Solution 的實際指令需先依產生的是 `.slnx` 或其他格式確認，不要在未看到實際檔案前猜測。

### 下一步三：成功執行空白網站

目標：

1. 用 VS Code 開啟專案根目錄。
2. 還原 NuGet 套件。
3. 執行空白 Blazor Web App。
4. 在瀏覽器成功開啟 localhost。
5. 確認 Hot Reload 與停止執行方式。
6. 確認 VS Code 的 Solution Explorer 與 Debug 流程。

### 下一步四：建立成交值排行功能目錄

空白網站成功後才建立：

```
```text
src/Invest.Web/Features/TradingValueRanking/
├─ Components/
├─ Models/
├─ Services/
└─ SampleData/
```
```

同時建立必要的：

```
```text
src/Invest.Web/Domain/Stocks/
src/Invest.Web/Domain/StockGroups/
```
```

暫時不要建立：

- EF Core Migration。
- Google Sheets Client。
- TWSE Client。
- TPEx Client。

### 下一步五：確認 MVP 規格

正式寫計算前，需請使用者確認：

1. 第一版是否採 3 個測試族群。
2. 是否使用約 30 個交易日測試資料。
3. 第一版期間是否先做 5／10／20 日。
4. 是否暫緩 60 日，直到有至少 120 個交易日。
5. 預設排行模式是成交熱度或資金加速。
6. 上漲家數比是否先採期間起點至終點。
7. 族群期間報酬率是否先不顯示。
8. 排行表最終欄位。
9. 前期是否固定採緊鄰同長度區間。

### 下一步六：建立假資料與純計算服務

先建立：

- 股票資料。
- 族群資料。
- 族群成員。
- 30 個交易日成交值及收盤價。
- 全市場每日成交值。

再建立純 C# 計算服務，依序完成：

1. 交易日排序。
2. 本期及前期切割。
3. 平均每日成交值。
4. 成交值增減率。
5. 市場成交比。
6. 排名。
7. 排名變化。
8. 上漲家數比。
9. 前三大集中度。
10. 成分股貢獻。

計算服務應與 Razor UI 分離，並可由單元測試直接呼叫。

### 下一步七：完成互動表格

完成：

- 期間按鈕。
- 排行模式切換。

- 表頭排序。
- 正負變化格式。
- 族群明細頁。
- 成分股貢獻表。
- 資料不足提示。

公式經測試確認後，才進入 SQLite、Google Sheets 及正式行情 API。

---

## 12. 給下一個 ChatGPT 的執行指示

1. 使用繁體中文回答。
2. 將使用者視為：
  - 熟悉 C# / WPF 的工程師。
  - 但不熟悉 macOS Terminal、VS Code、Blazor、EF Core 與網頁開發流程。
3. 不要重新將專案導向完整概念股、知識圖譜、新聞系統或資產配置。
4. 當前第一優先是：

```
```text
台股族群成交值排行榜
以及 5 / 10 / 20 / 60 個交易日的排行與成交熱度變化
```
```

5. 專案根目錄固定為：

```
```text
/Users/frankchiang/Desktop/Application/Code/Project/Invest
```
```

6. 第一版架構採：

```
```text
一個 Repository
一個 Solution
一個 Blazor Web Project
以 Features 目錄切分多個業務功能
```
```

7. 成交值排行集中於：

```
```text
src/Invest.Web/Features/TradingValueRanking
```
```

8. 不要把大量檔案隨意放在專案根目錄、`Components/Pages` 或單一 `Services` 資料夾。

9. 第一版技術組合：

```
```text
macOS
VS Code
C# Dev Kit
.NET 10
Blazor Web App
Interactive Server
ASP.NET Core
Entity Framework Core
SQLite
Google Sheets API
Git / GitHub
```
```

10. 不要一開始加入：

```
```text
React
TypeScript
Docker
PostgreSQL
Redis
微服務
盤中即時行情
AI 綜合評分
新聞系統
複雜 Clean Architecture
```
```

11. 開發順序必須採小步驟：

```
```text
確認環境
→ 建立 Solution
→ 跑起空白網頁
→ 建立功能目錄
→ 建立測試資料
→ 驗證計算公式
→ 建立互動排行榜
→ 建立單元測試
→ SQLite
→ Google Sheets
→ TWSE／TPEX
→ 歷史回補
→ 每日更新
```
```

12. 目前最適合接續的工作不是直接寫排行榜，而是先取得環境檢查結果。

13. 請先要求或確認以下輸出：

```
```bash
```

```
pwd
sw_vers -productVersion
uname -m
dotnet --version
dotnet --list-sdks
git --version
code --version
...
```

14. 每次提供指令時都要說明：

- 在哪裡輸入。
- 指令用途。
- 預期結果。
- 哪些結果屬正常。
- 發生錯誤時要貼出哪些內容。

15. 不要一次丟出整個專案的所有程式碼。應完成一個可驗證步驟，再繼續下一步。

16. 尚未確定的演算法不可自行宣稱已定案，尤其是：

- 族群報酬率算法。
- 預設排行模式。
- 預設觀察期間。
- 上漲家數比定義。
- 60 日是否進入第一版。
- 前期是否固定使用緊鄰同長度區間。
- TWSE／TPEX 實際 Endpoint。
- 每日更新方式。

17. 上一個回覆提出但尚未獲使用者確認的暫定 MVP 規格包括：

- 3 個測試族群。
- 每群 5~10 檔股票。
- 約 30 個交易日。
- 第一階段先做 5／10／20 日。
- 60 日延後。
- 兩種排行模式：成交熱度、資金加速。
- 成交熱度依平均每日成交值排序。
- 資金加速依成交值增減率排序。
- 上漲家數比暫採期間起點至終點。
- 暫不顯示族群期間報酬率。

以上內容在正式實作前應再次向使用者確認。

18. 不要宣稱目前已完成：

- Blazor 專案。
- Git Repository。
- SQLite。
- EF Core。
- Google Sheets API。
- TWSE／TPEX 串接。

- 排行榜頁面。
- 任何正式計算服務。

19. 目前實際狀態只有：

- 需求整理。
- 技術選型。
- 目錄規劃。
- 指標公式草案。
- 資料表草案。
- 環境檢查指令已提供，但尚未取得結果。

20. 下一個 ChatGPT 應從環境檢查結果接續，不要重新進行大範圍架構討論，除非檢查結果顯示既定技術無法使用